

A Self-Iteration Code Generation Method Based on Large Language Models

Tianyou Chang
College of Intelligence and Computing
Tianjin University
Tianjin, China
cty_2022@tju.edu.cn

Guodong Fan
College of Intelligence and Computing
Tianjin University
Tianjin, China
guodongfan@tju.edu.cn

Shizhan Chen
College of Intelligence and Computing
Tianjin University
Tianjin, China
shizhan@tju.edu.cn

Zhiyong Feng*
College of Intelligence and Computing
Tianjin University
Tianjin, China
zyfeng@tju.edu.cn

Abstract—Although large language models (LLMs) have demonstrated impressive performance in code generation, they still face challenges when dealing with complex code generation tasks. In the software development process, humans often refine complex tasks iteratively and continuously modify and improve them. Inspired by this, we propose a self-iteration code generation framework based on large language models like ChatGPT. To realize this idea, we incorporated software development methodologies into the self-iteration framework. We introduced four roles into each cycle, including analyst, designer, developer, and tester. Each role performs different tasks during the self-iteration cycle, with analyst and designer continuously improving requirements analysis and task design based on the testing feedback provided by tester, while developer are responsible for refactoring or modifying code until it passes testing, concluding the entire self-iteration process. We conducted extensive experiments on multiple benchmarks. The experimental results indicate: (1) The code generated by the self-iteration framework achieves up to a 21.3% relative improvement in Pass@1 compared to direct code generation. (2) The self-iteration framework also exhibits strong generalization performance, enhancing code generation quality for different large language models.

*“Failure is simply the opportunity to begin again,
this time more intelligently.”*

- Henry Ford

Index Terms—Iteration, Code generation, Large language models

I. INTRODUCTION

As one of the most important research areas in programming theory, code generation refers to the process of transforming specifications or descriptions at a high-level abstraction into executable computer program code using automated tools or programming techniques. In recent years, code generation has made remarkable progress in both the academic and industrial sectors [1]–[3]. It's worth noting that large language models

(LLMs) have demonstrated impressive capabilities in the field of code generation [4]–[6].

However, as programming demands become increasingly complex, both human programmers and LLMs face a series of complex challenges and difficulties. For instance, in the process of software development, especially when dealing with intricate programming issues, initial solutions often prove to be imperfect. Therefore, multiple adjustments and improvements are needed to continuously optimize and refine them. Iteration refers to the method of dividing a task into multiple cycles or stages during problem-solving, product development, or project completion. Within each cycle or stage, a series of steps is repeated to progressively enhance and refine the final outcome.

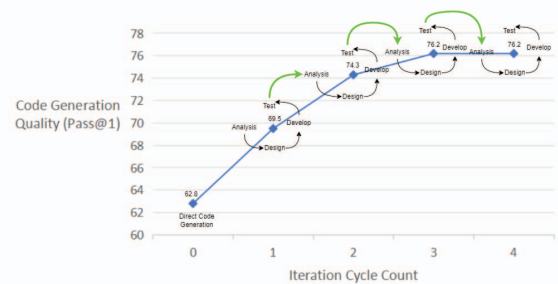


Fig. 1: The improvement in code quality generated by the self-iteration method based on ChatGPT on the HumanEval benchmark. (measured using the pass@1 metric)

The benefits of iteration are multifaceted: (1) Complex code generation problems are often not easily solvable in one go. Through iteration, the process of generating code can be refined and improved incrementally. (2) Iteration breaks down complex tasks into smaller, manageable parts, reducing the

*Corresponding Author

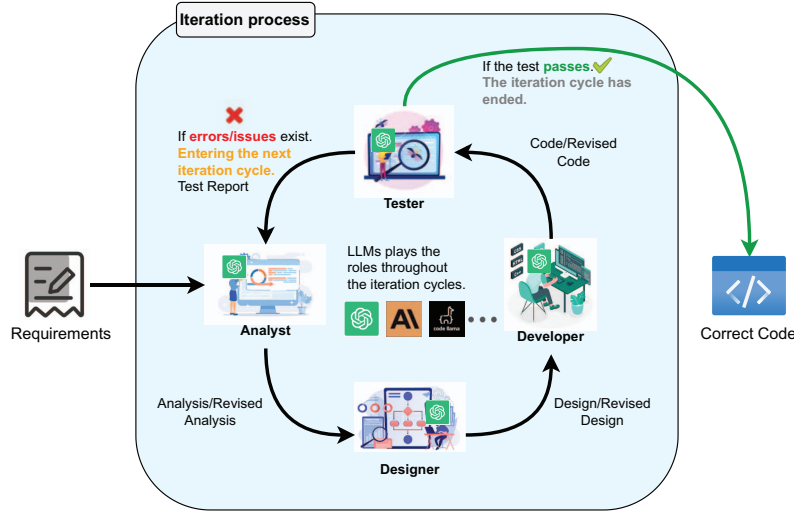


Fig. 2: The flowchart of the self-iteration framework.

difficulty of the task. Using an iterative approach to address each component also mitigates the risk of project failure. (3) With each round of iteration, team members accumulate more knowledge and experience, leading to a better understanding of the essence of the problem. This contributes to improving both the efficiency and quality of task completion.

Large language models excel in every subtask of the software development process, laying the foundation for a step-by-step approach to iteration. Furthermore, large language models can gain a deeper understanding of user requirements through interaction, enabling more precise analysis, refinement of requirements, and subsequent optimization. Finally, these large language models can continuously learn from and analyze existing data to identify and rectify potential errors and biases.

For this purpose, we propose a self-iteration framework to address complex problems while simultaneously enhancing the quality of code generation. In our self-iteration framework, we draw inspiration from the iterative process in software development methodologies [7], [8]. Our self-iteration framework comprises n cycles, each consisting of four steps: 1. Requirement analysis. 2. Structured design. 3. Code writing. 4. Testing feedback. Each subsequent cycle updates the requirement analysis or redesigns the solution based on the testing feedback from the previous cycle. Within these four steps of the iterative cycle, we instantiate four roles using ChatGPT, playing the roles of analyst, designer, developer, and tester. We applied the self-iteration framework to large language models, such as ChatGPT, and achieved impressive results. The improvement in code quality achieved through our self-iteration approach based on large language models is depicted in Fig. 1.

Our main contributions can be summarized as follows: (1) We introduced a self-iteration framework with continuous self-improvement capabilities, enabling large language models

(LLMs) to iteratively enhance themselves for code generation tasks with complex requirements. (2) We instantiated a team composed of four ChatGPT roles (analyst, designer, developer, and tester) within each iteration cycle, each responsible for their respective tasks and self-optimization. (3) Based on the self-iteration framework, ChatGPT (GPT-3.5) achieved top performance in multiple code generation tasks, and the self-iteration framework remains effective for other large language models.

II. SELF ITERATION

The flowchart of the self-iteration framework we propose is depicted in Fig. 2.

We have introduced the iterative method from software engineering methodologies into our self-iteration framework to enhance the quality of code generation. Specifically, the iterative process consists of n cycles, each comprising four stages: analysis, design, development, and testing. We have instantiated four roles, each assuming the roles of an analyst, designer, developer, and tester in each iteration cycle, respectively responsible for the analysis, design, development, and testing stages. These four roles are assigned the following tasks across different cycles, where n represents the cycle number:

1. Analyst: In the first iteration cycle (i.e., $n = 1$), the analyst is required to perform a brief analysis of the requirements provided by the user. They break down the given requirement x into several manageable sub-problems to facilitate the design team in devising solutions for these sub-problems. In subsequent iteration cycles (i.e., $n > 1$), the analyst needs to analyze the test reports from the first cycle and update or revise the requirement analysis document to reflect new requirements, changes, or optimizations.

2. Designer: In the first iteration cycle (i.e., $n = 1$), the designer must comprehend the requirements and analysis, and then formulate an initial solution workflow. They start with

an input-output structure and employ four fundamental structures—sequence, branching, recursion, and looping—while constructing the structured solution process. The details need to be described in natural language. In subsequent iteration cycles (i.e., $n > 1$), the designer is tasked with redesigning or improving the previously structured design document based on the updated requirement analysis results.

3. Developer: In the first iteration cycle (i.e., $n = 1$), the developer is tasked with initiating code writing based on the design document and requirement analysis. They implement the various modules and functionalities of the code, adhering to coding standards and best practices to ensure code quality. In subsequent iteration cycles (i.e., $n > 1$), developer need to modify existing code or begin writing new code based on the latest requirement analysis and structured design documents.

4. Tester: In the first iteration cycle (i.e., $n = 1$), the tester is responsible for creating a test plan and test cases based on the requirement analysis and structured design. They conduct functional tests to verify whether the code operates according to specifications. Any issues or errors are documented and reported, and a test report is prepared. In subsequent iteration cycles (i.e., $n > 1$), testers need to create test plans and test cases based on the updated requirement documents and design. They perform functional tests to ensure the quality of new features, record and report any issues or errors, and prepare a test report.

We assigned specific tasks to each role played by the LLM and informed each role of the tasks and responsibilities that involve interacting with other roles. Roles responsible for two consecutive stages can communicate with each other, and the tester's test reports are handed over to the next stage's analyst. The iteration process only terminates after the tester writes a passing test report or reaches the maximum iteration cycle n , as set by us.

III. EXPERIMENT SETUP

A. Benchmarks

Based on prior research [9], [10], we conducted experiments on two widely recognized code generation datasets: Human-Eval and MBPP. The Human-Eval [11] dataset comprises 164 programming questions, with an average of 9.6 test cases per programming question. The MBPP [12] dataset includes 500 programming questions in the test set, with an average of 3 automated test cases provided for each programming question.

B. Metrics

Pass@k: We evaluate the functional correctness of the top- k code generated by the model using an unbiased variant of pass@k [11] by executing test cases, where k represents the number of candidate codes that pass the tests. The formula is as follows:

$$\text{Pass @}k = \mathbb{E}_{\text{Problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (1)$$

In this paper, we primarily focus on pass@1, which represents the accuracy of individual generated code. This choice is motivated by the fact that, in real-world scenarios, people are more concerned with the correctness of single generated code.

CodeBLEU: CodeBLEU [13] is a variant of the BLEU score. Code contains rich syntactic and semantic structures that BLEU cannot effectively capture. Therefore, researchers introduced CodeBLEU to better assess the quality of automatically generated code. The formula is as follows:

$$\text{CodeBLEU} = \alpha \cdot \text{BLEU} + \beta \cdot \text{BLEU}_{\text{weight}} + \delta \cdot \text{Match}_{\text{ast}} + \zeta \cdot \text{Match}_{\text{df}}. \quad (2)$$

C. Baselines and Settings

Both artificial intelligence companies and open-source communities have trained multiple well-performing code generation models. We used several popular code generation models as baselines, and our baseline includes the following models: (1) GPT3.5&4 [14], (2) CodeX [11], (3) CodeT5+ [15], (4) CodeGeeX [10], (5) LLaMa [16], (6) PaLM Coder [17], and (7) CodeGen [3]. For these baseline models, we used NL descriptions, function signatures, and common test cases as input prompts. In the experiments involving ChatGPT, we utilized the gpt-3.5-turbo API. To enhance output stability, we set the temperature to 0 and limited the maximum number of iterations to 3.

IV. EXPERIMENTAL RESULTS

We assess the self-iteration approach by addressing 4 questions:

A. RQ1: How does the performance of the self-iteration method compare to the baseline method in code generation?

In the first research question, we applied the self-iteration approach and baseline models to two benchmark tests.

Evaluation: We applied the self-iteration approach using GPT-3.5 and compared it with other baseline models (Section III-C). We measured the quality of code generation using the standards of pass@1 and CodeBLEU (Section III-B).

Results: The primary results of the self-iteration approach, compared to other baseline models, are shown in Table I. The self-iteration approach achieved significantly better scores compared to other models. As seen in Table I, the self-iteration approach substantially improved the code generation capabilities of LLM (with GPT3.5 as an example). Regarding pass@1, the self-iteration approach based on ChatGPT (GPT3.5) exhibited substantial relative improvements on both benchmarks, 21.3% and 13.9%, respectively. It's worth noting that the self-iteration approach even outperformed GPT-4 on HumanEval.

To further understand the effectiveness of our approach, we studied the comparative performance of the self-iteration framework against a direct method. As shown in Fig. 3, we demonstrated this comparison using two examples of code

TABLE I: Comparison between the self-iteration method and other baselines.

Approach	HumanEval		MBPP	
	Pass@1	CodeBLEU	Pass@1	CodeBLEU
Code pre-trained models (also LLMs)				
CodeGeeX (13B)	22.9	23.1	24.4	18.4
LLaMa (65B)	27.7	-	-	-
CodeT5+ (16B)	30.9	26.3	36.8	28.1
CodeGen (16.1B)	34.6	22.8	36.6	24.5
PaLM Coder (560B)	36.0	-	47.0	-
CodeX (175B)	47.0	28.4	58.1	26.7
GPT4	75.0	-	-	-
ChatGPT	62.8	32.4	57.4	29.1
Self-iteration	76.2 (↑ 21.3%)	33.6 (↑ 3.7%)	65.4 (↑ 13.9%)	32.3 (↑ 8.2%)

generation—one produced directly and the other generated through the self-iteration framework. In these examples, the directly generated method often failed to comprehend the complex intent of the requirements, resulting in incorrect code. In contrast, the self-iteration framework minimized the risks of misunderstanding requirements and developing errors by refining complex tasks and continually improving them, leading to the generation of correct code.

B. RQ2: Do all roles play their part in each iteration cycle?

Evaluation. In this experiment, we studied the roles played by ChatGPT in the same iteration cycle, and the results are shown in Table II. The iteration cycle was set to 3 in this experiment. We considered that, during the iteration process, not only the developer but also a tester who writes test reports must be present. This is essential for correcting errors by the developer, ensuring that there is code correction and testing assessment in each iteration cycle, making the iteration meaningful. Therefore, we included the role of the tester in each experiment. The first row of Table II represents generating code directly with only developer receiving requirements.

TABLE II: The impact of different ChatGPT roles within the same iteration cycle (measured using the pass@1 metric).

Role	HumanEval	MBPP
Developer	60.9	54.8
+Tester	68.2 (↑ 11.9%)	59.8 (↑ 9.1%)
+Analyst +Tester	74.3 (↑ 22.0%)	62.4 (↑ 13.9%)
+Designer +Tester	72.6 (↑ 19.2%)	63.6 (↑ 16.1%)
+Analyst +Designer +Tester	76.2 (↑ 25.1%)	65.4 (↑ 19.3%)

Results. The experimental results demonstrate that the addition of each role during the iteration cycle contributes to the improvement of code generation quality. In both benchmarks, the team of Analyst-Designer-Developer-Tester achieved the best relative improvements compared to teams with only developer, with improvements of 25.1% and 19.3%, respectively.

C. RQ3: For large language models (using ChatGPT as an example), what is the optimal number of iterations?

Evaluation. To investigate the optimal number of iteration cycles, we employed a controlled variable approach by varying the number of cycles, as shown in Table III. A cycle count of 0 indicates that the iterative framework was not used, and

ChatGPT directly generated the code. A cycle count of 1 represents a single cycle, essentially a simplified version of the waterfall model. A cycle count of 2 corresponds to two rounds of the iterative framework, involving one round of modifications for analysis, design, and code, and so forth.

TABLE III: The impact of iteration cycle rounds (cycles) on the self-iteration framework (measured using the pass@1 metric).

The number of cycles	HumanEval	MBPP
0	62.8	57.4
1	69.5	61.2
2	74.3	63.2
3	76.2 (↑ 21.3%)	65.4 (↑ 13.9%)
4	76.2	65.2

Results. The experimental results indicate that the quality of the code experiences significant improvement even in the first iteration. Within the range of iteration cycles from 1 to 3, as the number of cycles increases, the code quality gradually improves, reaching its peak by the third cycle. However, by the fourth cycle, the quality of generated code no longer improves and may even slightly decline. Through case studies, we found that as the number of iterations reaches a certain value (in this experiment, 3), further iterations may introduce issues such as code redundancy and non-standard naming, leading to decreased code quality and increased bugs. In summary, this experiment demonstrates that more iterations are not necessarily better, and there might be diminishing returns after a certain point. For the iterative framework proposed for large language models (LLMs), the optimal number of iterations is determined to be 3.

D. RQ4: Does the self-iteration approach remain effective for large language models other than ChatGPT?

Evaluation. To investigate the generalizability of our self-iteration framework, we applied the self-iteration method to multiple popular large language models, including Codex-175B [11], Claude [18], PaLM Coder-540B [17], and LLaMa-65B [16]. Codex-175B and PaLM Coder-540B belong to specialized code generation models, while Claude and LLaMa-65B are chat-based large language models. In the experiments, we used the HumanEval benchmark and evaluated using the pass@1 metric (the percentage of successful single attempts).

Requirements	Direct Code Generation	Self-iteration Code Generation
Case 1 <pre>def iscube(a): """ Write a function that takes an integer a and returns True if this integer is a cube of some integer number. Note: you may assume the input is always valid. Examples: iscube(1) ==> True iscube(2) ==> False iscube(-1) ==> True iscube(64) ==> True iscube(0) ==> True """</pre>	<pre>import math def iscube(a): return math.isqrt(a)**3 == a</pre> ✗	<pre>def iscube(a): if a == 0: return True if a < 0: a = abs(a) root = round(a ** (1/3)) cube = root ** 3 if cube == a: return True else: return False</pre> ✓
Case 2 <pre>def search(lst): """ You are given a non-empty list of positive integers. Return the greatest integer that is greater than zero, and has a frequency greater than or equal to the value of the integer itself. The frequency of an integer is the number of times it appears in the list. If no such a value exist, return -1. Examples: search([4, 1, 2, 2, 3, 1]) == 2 search([1, 2, 2, 3, 3, 3, 4, 4, 4]) == 3 search([5, 5, 4, 4, 4]) == -1 """</pre>	<pre>def search(lst): counts = {} for num in lst: if num in counts: counts[num] += 1 else: counts[num] = 1 for key, value in counts.items(): if value >= key and key > 0: return key return -1</pre> ✗	<pre>def search(lst): frequencies = {} for num in lst: if num in frequencies: frequencies[num] += 1 else: frequencies[num] = 1 filtered_list = [] for num, freq in frequencies.items(): if freq >= num: filtered_list.append(num) if len(filtered_list) == 0: return -1 return max(filtered_list)</pre> ✓

Fig. 3: Two real cases from HumanEval, one utilizing self-iteration code generation and the other employing direct code generation, were used.

It's worth noting that ChatGLM2-6B [19] has a parameter size of less than 10B, and thus cannot strictly be categorized as a large language model.

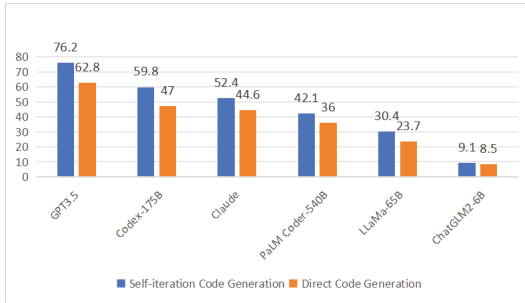


Fig. 4: Performance of the self-iteration framework on different large language models (measured by pass@1 on the HumanEval benchmark, which represents the percentage of successful single attempts).

Results. The performance of our self-iteration framework on different large language models is shown in Fig. 4. Our self-iteration framework significantly enhances the code generation capabilities of several popular large language models when the model's parameter size is equal to or greater than 65B. Furthermore, the magnitude of improvement increases with the model's parameter size. It is worth noting that for models with smaller parameter sizes, such as ChatGLM2 with a parameter

size of 6B, the improvement after applying the self-iteration framework is minimal. Overall, our self-iteration framework demonstrates good generalization performance and scalability, effectively improving the quality of code generated by various large language models.

V. CASE STUDY

We conducted a case study to evaluate the improvement in code quality brought about by our approach. We extracted cases where we generated code using our self-iteration framework on the HumanEval benchmark, and one such case is illustrated in Fig. 5. This case demonstrates the workflow and performance of our self-iteration code generation method. **In the first iteration:** (1) The analyst initially conducted a brief analysis of the requirement and decomposed it into several manageable subtasks. (2) The designer proposed an initial solution based on the requirement analysis and the decomposed subtasks. Subsequently, a detailed solution with structured features, including sequences, branches, loops, and recursion, was designed based on the initial solution. This structured approach facilitates the development of structured and logically clear code by the developer. (3) The developer wrote the code based on the subtasks decomposed by the analyst and the structured design solution provided by the designer. (4) The tester identified errors through test cases and reported them. The test report mentioned that the code did not convert the result into binary form and that it should not return the binary representation as a string. **In the second iteration:**

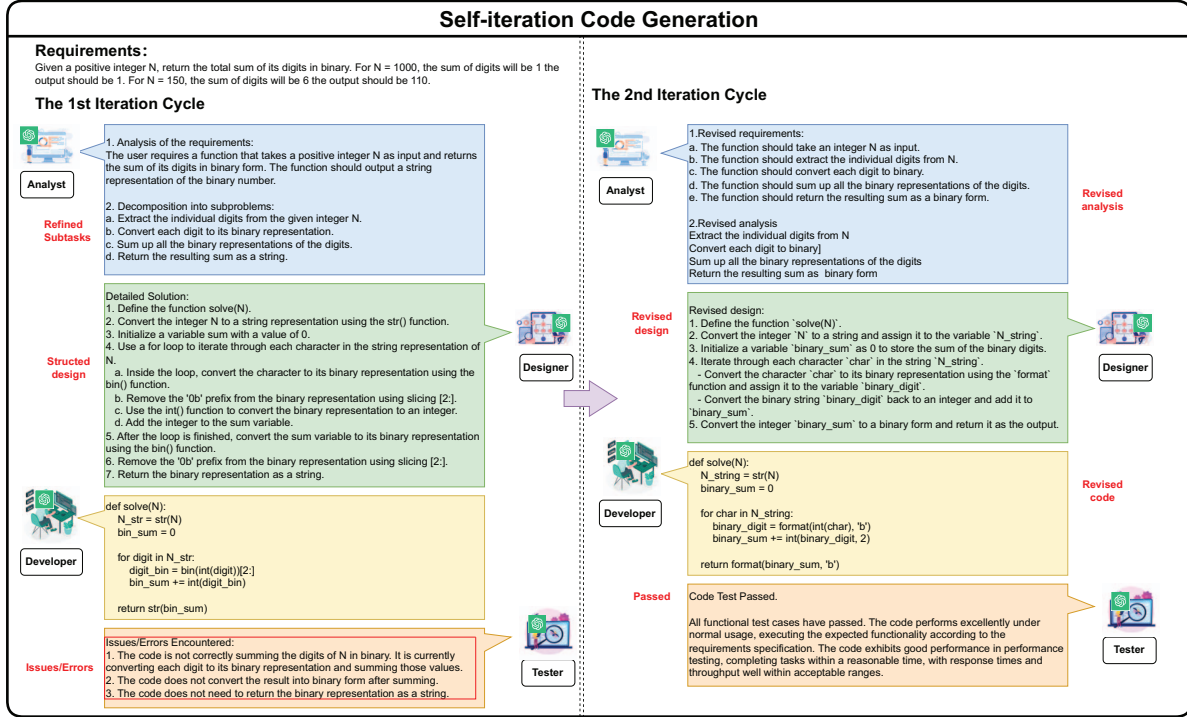


Fig. 5: Case study on the HumanEval benchmark. For clarity, we have omitted the inputs of the roles and focused on the progression of the iteration process.

(1) The analyst refined the decomposition of subtasks based on the test report. (2) The designer restructured the task design for subtasks after reviewing the error report and the modifications to subtasks. It is evident that the new structured design corrected the errors mentioned in the test report, such as the description of returning the result in binary form. (3) The developer wrote the code based on the new subtasks and design scheme. It is noticeable that the code corrected the errors that occurred in the first iteration, using “`return format(binary_sum, ‘b’)`” to return the result in binary form. (4) The tester evaluated the code generated in the second iteration and found that all tests passed, with no errors present. This concluded the entire self-iteration process. Throughout this process, we can intuitively observe that in the self-iteration process, the large language model successfully corrected the erroneous code through two rounds of self-iteration. This also demonstrates the effectiveness of our self-iteration framework.

VI. RELATED WORK

(1) Code Generation Traditional code generation methods focus on the hierarchical structure and syntax rules of source code. TreeGen [20] is a significant example that represents source code as a tree structure to better capture its hierarchical nature. Rabinovich et al. [21] introduced ASN, which maps source code to abstract syntax trees using the ADSL language, dynamically achieving more accurate code generation.

With the emergence of the Transformer framework [22] and the maturity of pretraining techniques, Large Language Models (LLMs) have grown in importance. LLMs have achieved significant success at various stages of code processing. CodeT5 [1], for instance, achieved impressive pretraining results by adopting four pretraining strategies consistent with code features. StarCoder [23] fine-tuned StarCoderBase, trained on licensing data from GitHub, to create a code generation model for the Python language. Code Alpaca [24] is a version of LLaMA [16] that underwent fine-tuning through self-instruct and ChatGPT [4] feedback, focusing on code generation tasks. WizardCoder [5] is a fine-tuned version of StarCoder, benefiting from ChatGPT feedback and using Code Alpaca’s dataset [25] as an initial dataset.

(2) Prompt Engineering Due to the large number of parameters in modern Large Language Models (LLMs) (e.g., Palm [17]: 540 billion parameters, LaMDA [26]: 137 billion parameters), fine-tuning or retraining these models has become challenging. Prompting is a commonly used technique in machine learning and natural language processing. It guides the model to generate specific types of outputs by providing explicit prompts or examples.

Chain of Thought [27] (CoT) is a recently proposed prompting technique. The main idea of Chain of Thought prompting is to combine a series of logical steps or thoughts into a prompt to guide the model in generating a sequence of coherent outputs. The model follows these steps in the prompt to generate text

or code, ensuring the coherence and rationality of the output. Jiang et al. [2] had large language models plan concise solution steps, and then the model gradually generated code guided by the preceding solution steps.

In this paper, we introduced the iterative process from software engineering into code generation. This allows large language models to perform task analysis and refinement during self-iteration cycles and continuously test and improve the generated code throughout the entire self-iteration process.

VII. CONCLUSION AND FUTURE RESEARCH

In this paper, we introduced a self-iteration framework aimed at gradually improving the capabilities of large language models to solve complex programming problems through self-iteration. We explored the potential of ChatGPT (GPT3.5) in iteratively generating code during the software development process. Specifically, based on the proposed framework, we had ChatGPT play four different roles within a single iteration cycle, continuously iterating through cycles to allow ChatGPT to self-improve and enhance the generated code. Our extensive research conducted on two benchmarks demonstrated the excellence of the self-iteration framework in terms of both pass@1 and human evaluation metrics. Applying the self-iteration framework to multiple large language models significantly improved the quality of code generation. This validates the effectiveness and generalizability of the self-iteration framework in the field of code generation. We believe that enabling models to self-iterate and continuously improve themselves is a crucial step in automated software development.

ACKNOWLEDGMENT

This work is supported by the National Natural Science Key Foundation of China grant No.62032016 and No.61832014.

REFERENCES

- [1] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 8696–8708.
- [2] X. Jiang, Y. Dong, L. Wang, Q. Shang, and G. Li, "Self-planning code generation with large language model," *arXiv preprint arXiv:2303.06689*, 2023.
- [3] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong, "Codegen: An open large language model for code with multi-turn program synthesis," in *The Eleventh International Conference on Learning Representations*, 2022.
- [4] OpenAI. ChatGPT. 2023. [Online]. Available: <https://openai.com/blog/chatgpt/>
- [5] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, and D. Jiang, "Wizardcoder: Empowering code large language models with evol-instruct," *arXiv preprint arXiv:2306.08568*, 2023.
- [6] Y. Dong, G. Li, and Z. Jin, "Codep: grammatical seq2seq model for general-purpose code generation," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 188–198.
- [7] C. Larman, *Agile and iterative development: a manager's guide*. Addison-Wesley Professional, 2004.
- [8] R. Ramsin and R. F. Paige, "Iterative criteria-based approach to engineering the requirements of software development methodologies," *IET software*, vol. 4, no. 2, pp. 91–104, 2010.
- [9] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "Codet: Code generation with generated tests," in *The Eleventh International Conference on Learning Representations*, 2022.
- [10] Q. Zheng, X. Xia, X. Zou, Y. Dong, S. Wang, Y. Xue, Z. Wang, L. Shen, A. Wang, Y. Liet al., "Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x," *arXiv preprint arXiv:2303.17568*, 2023.
- [11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [12] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Leet al., "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
- [13] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "Codebleu: a method for automatic evaluation of code synthesis," *arXiv preprint arXiv:2009.10297*, 2020.
- [14] OpenAI, "GPT-4 technical report," *CoRR*, vol. abs/2303.08774, 2023.
- [15] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, and S. C. Hoi, "Codet5+: Open code large language models for code understanding and generation," *arXiv preprint arXiv:2305.07922*, 2023.
- [16] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azharet al., "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.
- [17] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann et al., "Palm: Scaling language modeling with pathways," *arXiv preprint arXiv:2204.02311*, 2022.
- [18] Anthropic. Claude. 2023. [Online]. Available: <https://www.anthropic.com/index/introducing-claude>
- [19] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, and J. Tang, "Glm: General language model pretraining with autoregressive blank infilling," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 320–335.
- [20] Z. Sun, Q. Zhu, Y. Xiong, Y. Sun, L. Mou, and L. Zhang, "Treegen: A tree-based transformer architecture for code generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 05, 2020, pp. 8984–8991.
- [21] M. Rabinovich, M. Stern, and D. Klein, "Abstract syntax networks for code generation and semantic parsing," *arXiv preprint arXiv:1704.07535*, 2017.
- [22] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [23] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chimet et al., "StarCoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.
- [24] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," 2023.
- [25] S. Chaudhary, "Code alpaca: An instruction-following llama model for code generation," 2023.
- [26] R. Thoppilan, D. De Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Duet et al., "Llama: Language models for dialog applications," *arXiv preprint arXiv:2201.08239*, 2022.
- [27] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou et al., "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24 824–24 837, 2022.